INTE 22293 : Software Architecture and Process Models

In Class Activity - Software Architecture in Practice - 21st July 2026

Index No: IM/2023/114

Stage 1: Requirements and Problem Immersion

Actors

- Student acting as a seller, posts listings for textbooks, electronics and services
- Student acting as a buyer, browses, messages, pays and leaves reviews
- University email verification system, confirms a student is a genuine campus user
- Payment gateway, processes card or campus wallet payments
- Email/SMS provider, delivers notifications to users

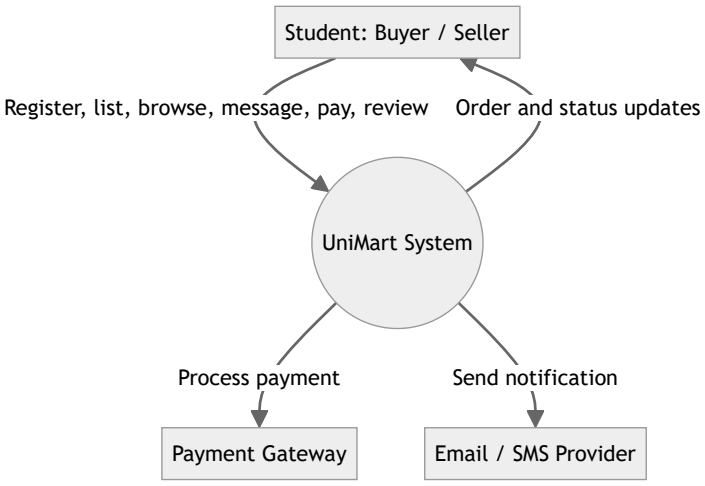
Functional Modules

- Registration and login with university email verification
- Catalog and listing management
- Search and browse/filter
- Messaging between buyers and sellers
- Orders and payments, including order status tracking
- Ratings and reviews
- Notifications for messages, price drops and order status changes

NFRs (traced to case study)

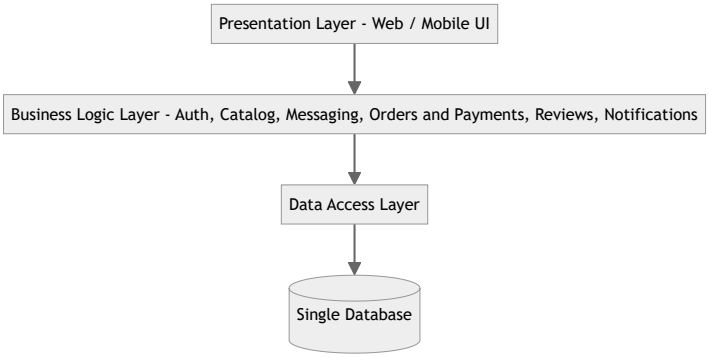
- **Availability during textbook rush:** the system "must work reliably during the two weeks at the start of each semester", so it needs high availability during those two predictable peak weeks.
- **Payment consistency:** "payments must never be lost or double-charged" is a strict consistency and reliability requirement on the payment flow.
- **Low cost, fast delivery:** "the three founders... need to be able to build and ship it fast with almost no budget" is a deployability and time to market constraint on the Year 0 architecture.

System Context Diagram (C4 Level 1)



Stage 2: Design the Monolith (Year 0 Architecture)

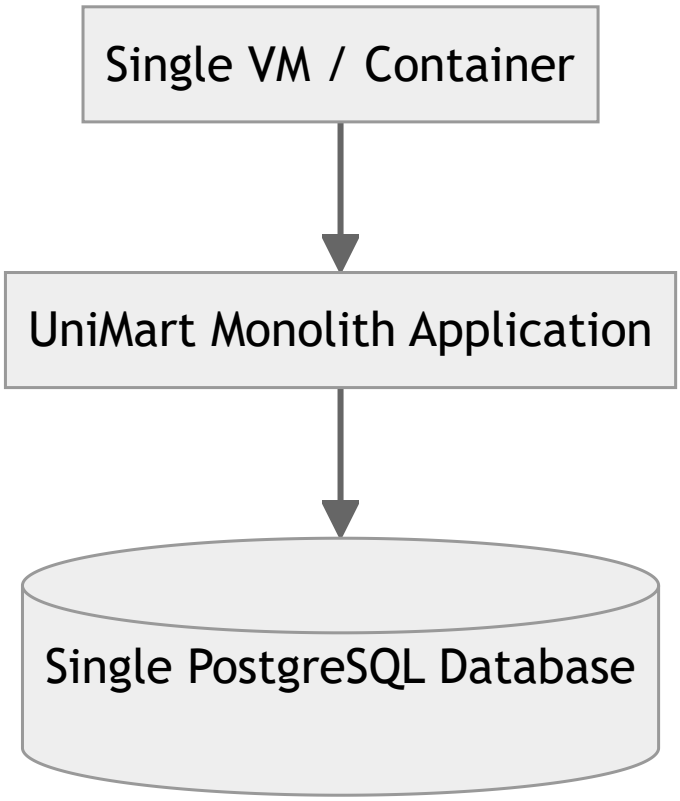
Layering Scheme



Internal Modules (within the single deployable)

- Auth: registration, login, email verification
- Catalog: listings, categories, photos
- Messaging: buyer/seller negotiation
- Orders/Payments: order lifecycle and payment processing
- Reviews: ratings and feedback after transaction
- Notifications: message, price drop and status alerts

Deployment Sketch



Technology Justification

A single Node.js/Express style monolith backed by one PostgreSQL database is the right call for the founding team, since it lets three students build, test and deploy one codebase without a dedicated ops person, and running one application and one database keeps hosting cost close to zero.

Reflection

1. If a bug in messaging crashes the server, catalog, payments, auth and everything else go down too, because all modules run inside the same process on the same machine, so one unhandled exception takes the whole application with it.
2. If textbook rush traffic hits only search, the only option is to scale the entire VM vertically or run more copies of the whole app horizontally, which is awkward because catalog, payments and every other module get duplicated even though only search actually needs the extra capacity.
3. A one line fix to reviews still has to pass through the same build, test and deploy pipeline as the rest of the app, so it could take hours rather than minutes despite being a trivial change.

Stage 3: The Growth Crisis, Pain Point Diagnosis

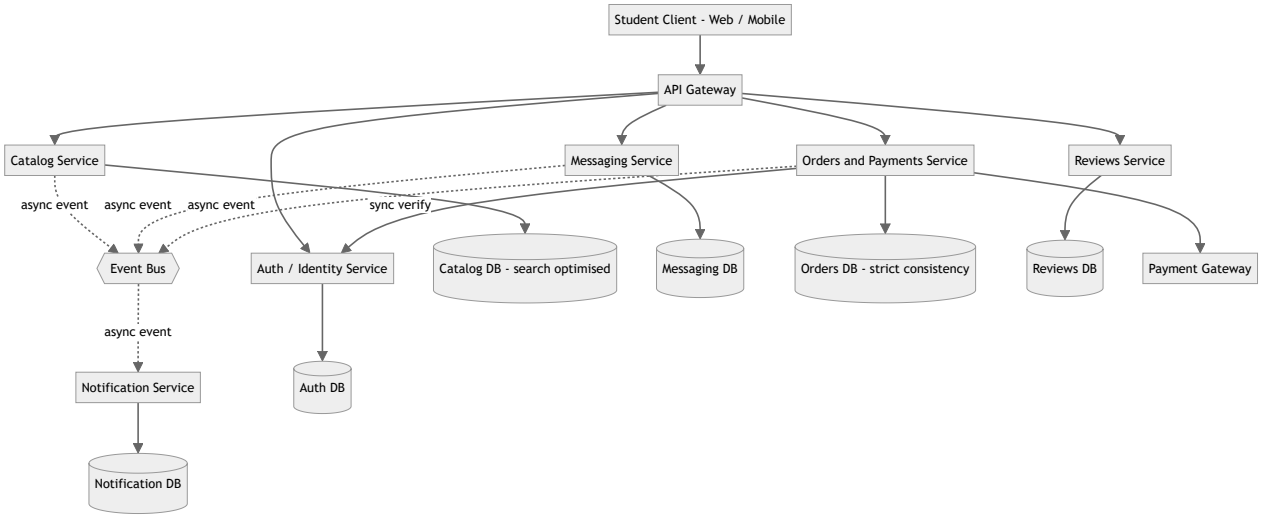
Pain Point	**Architectural Cause**
Search memory leak crashed the whole app, including payments mid transaction	Shared process/failure domain, all modules run in one process so one module's failure takes down the entire app
Payments wants a strict consistency database, Catalog wants a search optimised one, neither can move	Shared database, one schema/instance serves all modules so no squad can change engine without affecting the other three
A copy change to a listing page takes 3 days to reach production	Shared deploy pipeline, every change however small rides the same build, test and release cycle as payments code
Two squads shipped conflicting changes to the same file twice this month	Shared codebase, all four squads commit to one repository so unrelated features collide during merges

Stage 4: Redesign as Microservices

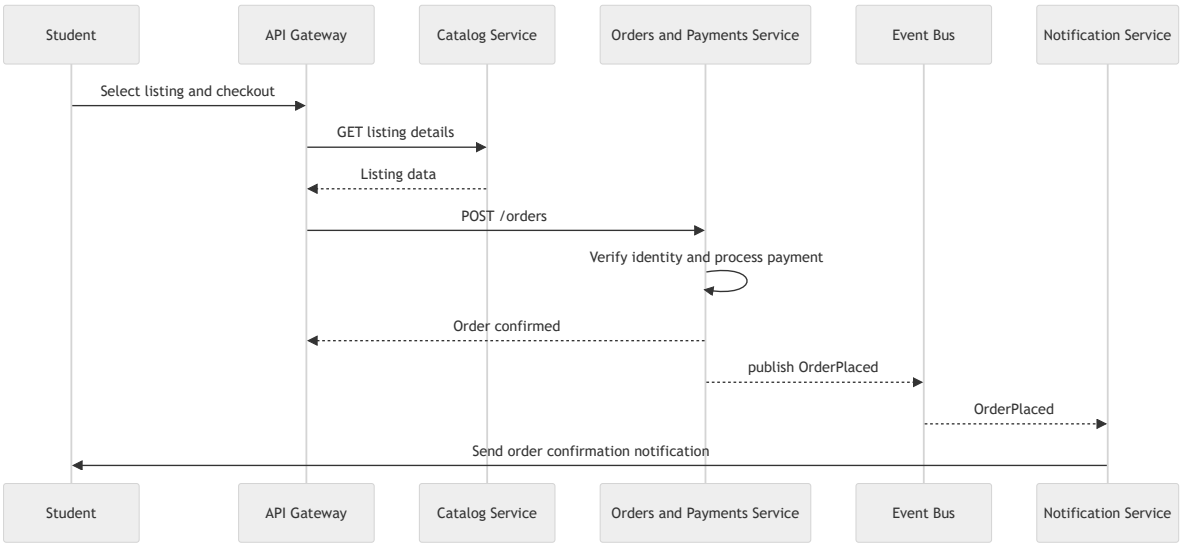
Service Boundaries

Service	Responsibility	Data Owned	Sample Endpoints	Communication
Auth/Identity	Registration, login and university email verification	User accounts, credentials, verification status	POST /auth/register POST /auth/login	Called synchronously by other services (REST) to verify identity, since an action must not proceed on an unverified user
Catalog	Create, update, search and filter listings	Listings, categories, photo metadata	POST /catalog/listings GET /catalog/listings?query=	Publishes ListingCreated and PriceDropped events asynchronously, since Notification being slow should not block a listing update
Messaging	Buyer and seller negotiation messages	Conversations, messages	POST /messaging/conversations/{id}/messages GET /messaging/conversations/{id}	Publishes NewMessageReceived asynchronously, same reasoning as Catalog
Orders and Payments	Process payment and track order status (posted, paid, completed)	Orders, payment transactions	POST /orders POST /orders/{id}/pay	Calls Auth synchronously to confirm buyer identity before charging; publishes OrderPlaced asynchronously to Notification
Reviews	Collect and display ratings after a completed transaction	Reviews, ratings	POST /reviews GET /reviews?sellerId=	Subscribes asynchronously to OrderCompleted, since prompting a review does not need to block order completion
Notifications	Send alerts for messages, price drops and order status changes	Notification preferences, delivery logs	GET /notifications/{userId} POST /notifications/preferences	Subscribes asynchronously to events from Catalog, Messaging and Orders/Payments

C4 Container Diagram



Sequence Diagram: Student Buys a Used Textbook



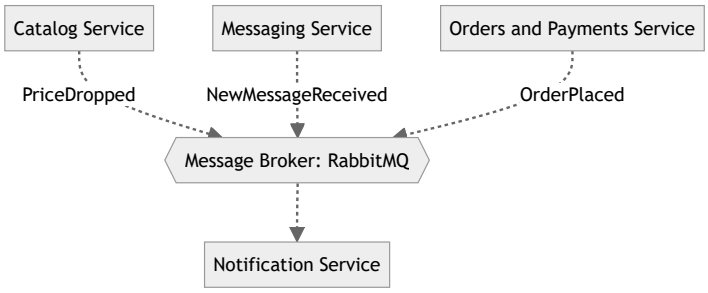
Trade Off Table

Dimension	What Got Better	What Got Harder
Deployability	Catalog can ship a listing page copy change without waiting on the payments release cycle	Six separate pipelines now need to be built and maintained instead of one
Fault isolation	A search memory leak in Catalog no longer takes down Payments, since they run as separate processes	Synchronous calls, such as Orders verifying identity with Auth, introduce network failure as a new failure mode
Data consistency	Payments can use a strict consistency financial database suited to its own needs, without Catalog's requirements holding it back	An order that spans Orders, Payments and Notification can no longer be one ACID transaction, so partial failures need extra handling
Operational complexity	Platform squad can now focus operations effort only on Auth and Notifications	The team now needs service discovery, per service monitoring and network failure handling across six deployables
Team autonomy	Catalog squad can adopt a search optimised database independently of Payments' consistency needs, solving the exact conflict from Stage 3	Squads must now agree on and maintain stable API contracts between services

Note: microservices are not a strict upgrade here. The real costs are distributed transactions across Orders, Payments and Notification, and the added need for service discovery and cross service monitoring that did not exist in the monolith.

Stage 5: Event Driven Notification

Message broker: RabbitMQ. Catalog, Messaging and Orders/Payments publish events without knowing who is listening, and Notification subscribes to the events it cares about and acts independently.



This buys decoupling, since Catalog, Messaging and Orders do not need to know Notification exists, and resilience, since Notification being slow or down during textbook rush does not block a purchase or a message from going through. It costs traceability, since following "what happened and when" across a producer, a broker and a consumer is harder than a single call stack, and it introduces eventual consistency, so a student could complete a purchase a few seconds before the confirmation notification arrives.

Take Home Extension

SOA vs Microservices: SOA typically relies on a heavier shared middleware layer, an Enterprise Service Bus, to route messages and enforce contracts centrally, and its services tend to be coarser grained, for example one "Order Management" service covering several sub functions. Microservices are deployed independently, are much finer grained, and prefer lightweight communication such as

REST or events without a central bus owning business logic, with each service strictly owning its own data store rather than sharing a common enterprise data model.

Serverless fit for UniMart: A nightly job that compiles a sales and top category report for the founders would fit well as a serverless function, since it runs briefly and on a schedule and does not need a permanently running service. The Messaging service would be a poor fit for serverless, since it needs to stay available and stateful for real time conversations.

Stage 6: Decision Memo

To the UniMart co-founders,

Building a monolith at Year 0 was the right call. With one semester, no budget and no dedicated ops person, one deployable app with one database let the three of you ship the full feature set fast and kept hosting and operational overhead near zero.

Growth to 60,000 users across 5 campuses exposed the cost of that choice. The search memory leak crashed the whole app because every module shares one process, so payments failed mid transaction and students were charged with no confirmation. Payments and Catalog now need different databases but share one, so neither squad can change it without affecting the other three. A one line copy fix now waits 3 days behind the payments pipeline, and two squads keep clashing in the same files.

Full microservices is not necessary yet. I recommend a partial decomposition: pull Orders and Payments out first, since it carries the hardest consistency requirement and the biggest blast radius, and keep Catalog, Messaging, Reviews and Notifications as a modular monolith until each squad's own pain justifies splitting further.

Expect distributed transaction handling and cross service monitoring to be the next problem.